🌐 ASL Teacher App – Detailed Project Structure & Purpose

📁 Root Directory

```
project-root/
├── backend/
├── frontend/
├── docker-compose.yml
├── Dockerfile
├── nginx.conf
├── vite.config.mjs
└── .env
```

These are **global files** that define how your frontend and backend interact and how the project runs both locally and in containers.

File/Folder	Purpose
backend/	Contains all backend logic — REST APIs, database interaction, and server configuration.
frontend/	React + Vite frontend that renders the ASL learning interface.
docker-compose.yml	Defines multi-container setup (backend + database). Makes it easy to start everything with one command.
Dockerfile	Defines the build process for the combined frontend + backend image. It first builds the frontend, then copies it into the backend's `public/` directory for serving.
nginx.conf	(Optional) Configuration for NGINX if you want to serve frontend through a reverse proxy — typically used in production to handle routing or load balancing.
vite.config.mjs	Vite's build configuration — controls how frontend is bundled and optimized for production.
.env	Holds environment variables (e.g., DB credentials, hostnames, ports) used by both backend and Docker.

⚙️ Backend Folder Structure

```
backend/
├── app/
│   ├── crud.py
│   ├── database.py
│   ├── main.py
│   ├── models.py
│   ├── schemas.py
│   └── requirements.txt
├── routers/
│   ├── auth.py
│   └── tests.py
├── server.js
├── package.json
```

```
├── .dockerignore
└── .env
```

🧠 Backend Explanation

File/Folder	Description
app/	Core backend logic folder — includes everything related to database, models, and CRUD operations.
app/crud.py	CRUD = Create, Read, Update, Delete operations. Contains database interaction functions used by routes (e.g., save results, fetch history).
app/database.py	Manages database connections to PostgreSQL using connection strings or SQLAlchemy (depending on setup). Initializes database engine and sessions.
app/main.py	Entry point for FastAPI (if you're using it alongside Express for specific routes). Runs the backend logic before integration with `server.js`.
app/models.py	Defines database models/tables (like User, TestResults, Gestures). Mapped to PostgreSQL tables.
app/schemas.py	Defines Pydantic schemas (if using FastAPI) or JSON schemas for request/response validation.
app/requirements.txt	Python dependencies list (used if running Python backend components).
routers/	Contains modular route files (API endpoints).
routers/auth.py	Authentication endpoints — login, register, token verification, etc.
routers/tests.py	Routes related to tests — uploading results, fetching user test history, scoring, etc.
server.js	**Main backend server** using Node.js + Express. Starts HTTP server, defines routes, connects to PostgreSQL, and serves built frontend from `public/`.
package.json	Node.js metadata and dependencies (Express, pg, dotenv, etc.).
.dockerignore	Excludes unnecessary backend files from Docker builds (like `node_modules`, `.env`, logs).
.env	Backend-specific environment file (DB credentials, JWT secrets, etc.). These are automatically picked up by Docker or Render.

🎨 Frontend Folder Structure

```
frontend/
├── src/
│   ├── components/
│   │   ├── camerafeed.jsx
│   │   ├── gesturetrainer.jsx
│   │   ├── learnpage.jsx
│   │   ├── login.jsx
│   │   ├── resultcontext.jsx
│   │   ├── resultspage.jsx
│   │   ├── testflow.jsx
```

```
│   │   └── ttsfeedback.jsx
│   ├── app.jsx
│   ├── main.jsx
│   └── styles.css
├── package.json
├── .dockerignore
└── .env
```

💻 Frontend Explanation

File/Folder	Description
src/	Contains all React source code.
src/components/	Modular components for the app UI — each focusing on one key function.
camerafeed.jsx	Captures real-time webcam input, detects hand gestures using the camera. May use TensorFlow.js or MediaPipe Hands for gesture tracking.
gesturetrainer.jsx	Displays gestures for the user to learn and practice. Provides visual feedback during training.
learnpage.jsx	Page showing structured ASL learning modules and progress tracking.
login.jsx	User authentication interface — login/signup. Communicates with `/auth` routes on the backend.
resultcontext.jsx	Context API wrapper for managing global state of test results (so all components can access current user's progress or scores).
resultspage.jsx	Displays performance analytics, user scores, and uploaded results history from backend.
testflow.jsx	Handles complete testing flow — instructions → capture → evaluate → upload → feedback.
ttsfeedback.jsx	Uses Text-to-Speech feedback to guide or correct user gestures.
app.jsx	The main React app structure — defines routing (React Router) and shared layout.
main.jsx	Entry point for rendering React into DOM. Also initializes API base URL for backend calls.
styles.css	Global styling, usually Tailwind + custom CSS overrides.
package.json	Contains React dependencies (Vite, Tailwind, Axios, React Router, etc.) and build scripts.
.env	Frontend environment variables like `VITE_API_URL=http://localhost:5000/api` (must start with `VITE_` for Vite).
.dockerignore	Excludes unnecessary frontend files (like `node_modules`, `.env.local`, `.DS_Store`) from Docker builds.

🐳 Docker-Related Files

File	Purpose
Dockerfile	Multi-stage build file. Builds frontend → copies built assets into backend → runs Node server to serve everything.
docker-compose.yml	Runs backend and PostgreSQL in separate containers for local development. Handles networking between services.
nginx.conf	Optional — if you want NGINX to serve frontend statically and proxy backend API requests. Useful for production scaling or HTTPS setup.
.env (root)	Global environment file for both backend and frontend containers (DB, ports, API base URL).

🧩 How It All Works Together

1. **Frontend Build**
   - Vite builds all React files into `/frontend/dist`.
   - These static files are copied into `/backend/public` by Docker.
   - When backend starts, Express serves these files for all routes.
2. **Backend API**
   - `/api/auth` → handles authentication (`auth.py`).
   - `/api/results` → handles uploading & fetching test results (`tests.py`).
   - `/api` routes connect to PostgreSQL through `database.py` and `crud.py`.
3. **Database Layer**
   - PostgreSQL container stores user data, gesture results, and training progress.
4. **Docker Workflow**
   - One command (`docker-compose up --build`) builds frontend + backend + DB.
   - Containers communicate internally on the Docker network.
5. **Deployment (Render or Cloud)**
   - The same Dockerfile builds a single deployable image.
   - Render injects environment variables from your dashboard.
   - The container automatically starts by running `node server.js`.

🌐 Final Deployment Integration Guide for Render

Your backend (Node.js + Express + PostgreSQL) is now hosted at:

🔗 **https://asl-teacher-app-11.onrender.com**

⚙️ 2. Backend Configuration on Render

Render Environment Variables

In your Render **Dashboard → Service → Environment** section, add the following:

For local testing (before deploying):

```
PGUSER=postgres
```

```
PGPASSWORD=123456
PGHOST=host.docker.internal
PGDATABASE=asl1_db
PGPORT=5432
PORT=5000
```

🐬 4. Dockerfile Reminder

Ensure your final Dockerfile uses these `ENV` defaults:

```
ENV PGUSER=${PGUSER}
ENV PGPASSWORD=${PGPASSWORD}
ENV PGHOST=${PGHOST}
ENV PGDATABASE=${PGDATABASE}
ENV PGPORT=${PGPORT}
ENV PORT=5000
```

☑ How Everything Works After This

1. User visits your site (frontend on Render or Netlify).
2. Frontend makes requests to:
3. `https://asl-teacher-app-11.onrender.com/api/results`
4. `https://asl-teacher-app-11.onrender.com/api/auth`
5. Backend connects to your PostgreSQL on Render using the environment variables.
6. Uploads and test results are saved and fetched successfully.

1 Dockerfile Setup

You already have a **multi-stage Dockerfile**:

- **Stage 1: Frontend Build**

```
FROM node:22-alpine AS frontend-build
WORKDIR /frontend
COPY frontend/package*.json ./
RUN npm install
COPY frontend/ ./
RUN npm run build
```

☑ Builds your Vite React frontend and outputs static files.

- **Stage 2: Backend Build**

```
FROM node:22-alpine AS backend
WORKDIR /app
COPY backend/package*.json ./
RUN npm install
COPY backend/ ./
RUN mkdir -p public
COPY --from=frontend-build /frontend/dist ./public
```

```
ENV PGUSER=${PGUSER}
ENV PGPASSWORD=${PGPASSWORD}
ENV PGHOST=${PGHOST}
ENV PGDATABASE=${PGDATABASE}
ENV PGPORT=${PGPORT}
ENV PORT=5000
EXPOSE 5000
CMD ["node", "server.js"]
```

☑ Copies built frontend into backend `public` folder.
☑ Sets environment variables for PostgreSQL.
☑ Starts Express backend.

---

2 docker-compose.yml

You can manage **backend, frontend, and Postgres** using Docker Compose:

```
version: "3.9"

services:
  db:
    image: postgres:18
    container_name: asl-postgres
    restart: always
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: 123456
      POSTGRES_DB: asl1_db
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  app:
    build: .
    container_name: asl-app
    ports:
      - "5000:5000"
    depends_on:
      - db
    environment:
      PGUSER: postgres
      PGPASSWORD: 123456
      PGHOST: db
      PGDATABASE: asl1_db
      PGPORT: 5432

volumes:
  pgdata:
```

☑ This allows your backend to connect to Postgres **inside Docker** (`PGHOST=db`).
☑ Port mapping exposes backend on `localhost:5000`.

---

3 Build & Run Containers Locally

From your project root:

```
docker-compose up --build
```

- `--build` ensures Docker rebuilds images with latest changes.
- Containers start in order: `db` → `app`.
- Logs appear in your terminal; backend connects to Postgres automatically.

---

4 Entering Containers

- To inspect backend:

```
docker exec -it asl-app sh
```

- To check Postgres:

```
docker exec -it asl-postgres psql -U postgres -d asl1_db
```

---

5 Common Commands

Command	Purpose
`docker ps`	List running containers
`docker logs -f asl-app`	Follow backend logs
`docker-compose down`	Stop and remove containers
`docker-compose up --build`	Build and run all services
`docker exec -it <container> sh`	Access shell inside container

---

6 Debugging Tips

1. **DB connection errors**: Make sure `PGHOST` matches service name in `docker-compose.yml` (`db` in this case).
2. **Frontend not loading**: Ensure `COPY --from=frontend-build /frontend/dist ./public` exists.
3. **Port conflicts**: Check `netstat -ano | findstr :5000` and stop other services if needed.
4. **Env variables**: Do **not hardcode** passwords in code. Use `.env` for local and Render environment variables for deployment.

7️⃣ Optional: Running Backend Separately

If you only want to run backend (for development):

```
docker build -t asl-backend .
docker run -p 5000:5000 --env-file .env asl-backend
```

- `--env-file .env` passes environment variables to container.